AEROSPLAN Handbook

AI Multi-Agent Travel Planning System Architecture & Implementation Q&A

1. Core Concept & Multi-Agent Philosophy

Q1: What is the core problem this project solves and what is the target audience?

Planning a vacation requires travelers to manually check multiple portals for flights, hotels, daily activities, and budget constraints, then copy everything into a document. Aerosplan consolidates this experience into a unified dashboard. By typing a simple natural language prompt (e.g., 'A 4-day trip to Goa next week under 50k'), the system orchestrates multiple specialized AI agents to search the web, calculate costs, draft a personalized day-by-day itinerary, check budget constraints, and display a fully comprehensive, interactive travel plan.

Q2: Why did you build this as a Multi-Agent system rather than using a single LLM prompt with system instructions?

A single LLM prompt suffers from context pollution, high hallucination rates, and struggles when executing multiple tools sequentially. Splitting the process into a multi-agent system provides:
1. Focus & Specialization: Each agent (Flight, Hotel, Itinerary, Budget) has a dedicated system instruction and limited tool scope.
2. Deterministic Routing: If Flights or Hotels fail to yield API results, conditional routing redirects to specific fallback nodes instead of failing the entire session.
3. Controlled State Sharing: Agents pass structured updates through a shared State schema, ensuring clear data boundaries and reducing prompt length.
4. Cost & Performance Optimization: Heavy math is offloaded to a standard Python calculator (Budget Tool) instead of forcing the LLM to do arithmetic, which it frequently fails at.

2. LangGraph Graph Architecture & Orchestration

Q3: Why did you choose LangGraph over other frameworks like CrewAI or AutoGen?

LangGraph (by the LangChain team) was chosen because it focuses on graph-based state machines. While frameworks like CrewAI and AutoGen are highly autonomous (making them hard to predict and expensive due to infinite reasoning loops), LangGraph gives the developer fine-grained control. It allows us to explicitly define graph nodes (agents), edges (flow paths), and conditional routing functions. This ensures the workflow is predictable, fast, and structured, while supporting cycles and database persistence.

Q4: Explain the graph structure of Aerosplan. What are the nodes and how is state managed?

The graph is managed via a shared 'TravelState' defined using Python's TypedDict. The state persists fields like 'messages' (which uses a reducer operator.add to append chat history), 'user_query', 'flight_results',

'hotel_results', 'itinerary', 'budget_analysis', 'budget' limit, and 'llm_calls' count.

The nodes of the graph are:
- Flight Agent -> Queries the flights tool.
- Hotel Agent -> Queries the Tavily Search API for accommodations.
- Itinerary Agent -> Combines flights + hotels to construct the daily schedule.
- Budget Agent -> Computes total cost and suggests saving/upgrade recommendations.
- Final Response Agent -> Formats the complete package into clean Markdown layout.
- Fallback Agents -> Run if flight or hotel search returns zero matches.

Q5: How does conditional routing and fallback handling work in your graph implementation?

Conditional routing is implemented using 'graph.add_conditional_edges'. After the Flight Agent runs, a router function 'route_after_flights' checks the 'flight_results' field in the state. If it detects failure keywords (like 'unavailable' or 'no flights'), it returns the string 'no_flight_fallback'. The graph engine routes control to the 'no_flight_fallback' agent, which suggests backup transit (trains, buses, nearby airports) and then rejoins the main flow at the Hotel Agent. The same pattern is used for the Hotel Agent with a 'no_hotel_fallback' route.

3. Backend, SSE Streaming, and API Design

Q6: How is real-time progress update streaming implemented between the FastAPI server and React client?

We use Server-Sent Events (SSE) via FastAPI's 'StreamingResponse' returning 'text/event-stream'.
1. Backend: When a user hits '/api/plan', FastAPI triggers LangGraph using 'app.stream(..., stream_mode="updates")'. This generator streams state changes node-by-node. As each agent completes, we yield a structured event containing the agent name and its specific output (e.g. 'event: update\ndata: {...}\n\n').
2. Frontend: The React client uses the Fetch API's 'ReadableStream' reader to listen to the HTTP stream. A buffer parses the stream line-by-line, extracting the event type (start, update, complete, error). React updates its state variables in real-time, allowing the UI to render active agent progress (e.g., highlighting which agent is running) and stream text step-by-step.

Q7: Why did you choose Server-Sent Events (SSE) over WebSockets or long polling?

1. SSE vs. WebSockets: WebSockets are bidirectional and complex, requiring separate handshakes and connection management. Since our application only needs to stream data from the server to the client (one-way), SSE is lightweight, operates over standard HTTP, and automatically handles client disconnections/reconnections.
2. SSE vs. Polling: Polling is highly inefficient because the client must constantly send requests to check if the plan is ready. SSE pushes data instantly, reducing network latency and database queries.

Q8: How did you optimize the streaming payload to prevent performance lag or browser memory issues?

Instead of returning the entire LangGraph state structure (which contains nested message history, raw metadata, and large object trees), we map each node update to a clean, lightweight payload in 'api.py'. The payload only contains keys that the frontend needs (like 'flight_results', 'hotel_results', 'itinerary', or 'budget_analysis'). This reduces bandwidth by over 90% and prevents frontend parsing overhead.

4. Database & State Persistence (Memory)

Q9: How does the session memory work in Aerosplan? Where is it stored?

Aerosplan uses LangGraph's checkpointer mechanism with a 'PostgresSaver'. This Saver is backed by a PostgreSQL database and a connection pool ('psycopg_pool.ConnectionPool'). By compiling the graph with a checkpointer, LangGraph automatically saves state variables at every graph node step using a unique 'thread_id'. This allows a user to refresh their browser, close the app, or fetch their history, and retrieve their exact plan history using their 'thread_id' from the database.

Q10: Explain your connection pooling strategy and why it is critical here.

We use 'psycopg_pool.ConnectionPool(DATABASE_URL)' to manage database connections. Instead of establishing a new TCP connection for every single query (which takes time and limits scale), the pool maintains active, recycled connections. If multiple users hit the API concurrently, the pool assigns connections instantly.

This prevents database lockups, handles concurrent requests smoothly, and provides robust performance.

Q11: How does deleting a trip work on the database level?

FastAPI exposes a DELETE endpoint: '/api/plan/{thread_id}'. To cleanly remove a plan, the API connects to PostgreSQL and runs three SQL deletes on the LangGraph system tables using the 'thread_id':
1. DELETE FROM checkpoints WHERE thread_id = %s;
2. DELETE FROM checkpoint_blobs WHERE thread_id = %s;
3. DELETE FROM checkpoint_writes WHERE thread_id = %s;
This ensures there are no orphan data records in the database, preserving clean storage.

5. LLM Integration & Custom Tools

Q12: Why did you use Groq (Llama 3.3 70B) as your LLM instead of GPT-4 or Claude?

1. Extreme Speed: Groq runs open-source models using custom LPU (Language Processing Unit) hardware, delivering speeds of over 200 tokens/sec. This is crucial for multi-agent loops where several LLM steps run sequentially. Standard models would take 30-40 seconds, while Groq takes 10-15 seconds for the entire graph.
2. Cost Efficiency: Utilizing Llama 3.3 70B through Groq is highly cost-effective compared to commercial closed-source APIs.
3. Reasoning Power: Llama 3.3 70B features a 128k context window and is highly capable of parsing structured text, organizing itineraries, and analyzing budgets.

Q13: Explain how the budget calculation tool works. Why is it a hybrid of regex and LLM and how does it scale for multiple travelers?

LLMs are notoriously poor at math. To solve this, we implemented a hybrid model:
1. Regex & Scaling (Budget Tool): We extract the target budget from the user's query. Next, the tool parses the number of travelers from the query (e.g. '3 people' or 'couple' -> 2).
   - Flight scaling: Takes the lowest flight price and multiplies it by the traveler count.
   - Hotel scaling: Calculates rooms needed (1 room per 2 guests using math.ceil) and multiplies the per-room cost by the room count.
   - The tool then adds the scaled flight and hotel costs together.
2. LLM (Budget Agent): These exact numbers (e.g. estimated total: Rs. 33,000, limit: Rs. 50,000, status: under, travelers: 3, rooms: 2) are sent to the LLM. The LLM then writes the user-friendly description and recommends upgrades (if under budget) or cost-saving swaps (if over budget). This guarantees mathematical accuracy while maintaining natural language flexibility.

6. Frontend Engineering & Design System

Q14: What is the design style and typography choices of the React application?

The frontend uses a premium, responsive glassmorphic dashboard styled with Tailwind CSS.
1. Typography: Google Fonts Outfit for headings (contemporary, bold structure) and Plus Jakarta Sans for body text (highly readable at smaller screen sizes).
2. Styling: Uses a curated slate/indigo palette. Includes custom CSS helper classes for glassmorphic cards (e.g., 'glass-panel' and 'glass-card' using backdrop-filter: blur) and a subtle background radial gradient that moves fluidly on screen sizes.
3. Real-time Indicators: Features a custom stepper indicator that lights up in real-time depending on which agent (Flight, Hotel, Itinerary, Budget) is executing.

Q15: How is the history sidebar integrated with the state?

On page load, React fetches the trip history (thread IDs, queries, budgets, and completed statuses) and renders them in a collapsible sidebar using Lucide-react icons. Clicking any past trip queries the GET

'/api/plan/{thread_id}' endpoint. The backend retrieves the compiled state values from database checkpointer tables, and React updates its local state to render the saved itinerary, budget analysis, and flight details instantly, without triggering new agent executions or LLM costs.

7. Session Management & Database Portability

Q16: How does Aerosplan handle user sessions and keep history segmented per user without standard email/password authentication?

For lightweight portability and convenience in self-hosted and presentation environments, Aerosplan implements a client-persisted UUID mapping pattern:
1. Frontend UUID Generation: On initial page load, the React client checks localStorage for 'aerosplan_user_id'. If not present, it generates a unique random alphanumeric identifier and saves it.
2. Session Thread Mapping: Every trip planning request (/api/plan) sends this user_id alongside the search query. The backend records the mapping in a 'user_trips' junction table.
3. Filtered History Retrieves: When loading the history sidebar, the client calls GET /api/history?user_id=usr_... . The backend filters the database checkpoints to return only the thread_id records that belong to that specific user_id.

Q17: How does the backend prevent connection crashes if a developer running Aerosplan does not have a PostgreSQL database server set up?

We implemented an automatic database connection fallback mechanism in main.py:
1. Connection Test: Upon backend initialization, the system tries to establish a brief test connection to the PostgreSQL URL (configured in .env) with a short 2-second timeout.
2. SQLite Fallback: If the connection fails (e.g., PostgreSQL server is offline or not installed), the system gracefully catches the error and instantiates a local, persistent SQLite database (aerosplan_memory.db) using LangGraph's native SqliteSaver checkpointer.
3. Unified Interface: We encapsulate these database differences within a custom DatabaseManager class, exposing uniform methods like get_user_threads(), record_user_thread(), and delete_thread(). This allows the API code to run without modification regardless of the database technology, preventing startup crashes and 'fetch errors'.